

Ans 1) Time Complexity is the amount of time an algorithm takes to run as the input size (n) increases.

Ans 2) Space Complexity is the amount of memory (RAM) an algorithm uses as the input size (n) increases.

It includes:

- Input Space (memory for the input)
- Auxiliary space (extra memory used by the algorithm)

Ans 3) Asymptotic notations describes how an algorithm's time or space complexity grows as the input size (n) becomes very large.

① Big O (O) - Worst Case

- Represents the maximum time an algorithm can take.
- Gives the upper bound of complexity.

② Omega (Ω) - Best case

- Represents the minimum time an algorithm can take.
- Gives the lower bound of complexity.

- ③ Theta (θ) - Average / Exact Bound
- Represents the tight (exact) bound when the algorithm grows at the same rate in both upper and lower bounds.
 - Used when the running time is consistently proportional.

eg: Traversing an array always requires visiting all elements $\rightarrow O(n)$

Ans 4)

I) BEST CASE

- Minimum time an algorithm takes.
- Happens in the most favorable situation.

eg: Time complexity: $O(1)$

II) AVERAGE CASE

- Expected time for a typical input.
- Represents the algorithm's performance on average.

eg: Time complexity: $O(n)$

III) WORST CASE

- Maximum time an algorithm takes.
- Happen in the least favorable situation.

eg: Time complexity: $O(n)$

Ans 5) Code Patterns
 Single Statement
 One loop
 Two nested loops
 Consecutive loops
 Ignore constants
 Keep highest term
 Halving / Doubling loop

Time Complexity
 $O(1)$
 $O(n)$
 $O(n^2)$
 add, then simplify
 $O(1), O(n), O(n^2)$
 eg $O(n^2 + n) = O(n^2)$
 $O(\log n)$

Ans 6) loop Pattern
 One loop
 Two nested loop
 consecutive loop

Time complexity
 $O(n)$
 $O(n^2)$
 $O(n)$

Ans 7) Nested loop type
 2 nested loop (max n)
 Inner loop $j \leq i$
 Outer n, Inner na

Time complexity
 $O(n^2)$
 $O(n^2)$
 $O(nm)$

Ans 8) loop Pattern
 $i^* = 2$
 $i / = 2$
 search space halves each
 step

Time complexity
 $O(\log n)$
 $O(\log n)$
 $O(\log n)$

(4)

Ans a)
 Memory Used
 few variables only
 one extra array of size (m)
 in recursive calls

Space Complexity
 $O(1)$
 $O(n)$
 $O(n)$

MODULE 1

Time Complexity: All the patterns shown are $O(n^2)$ because they are nested loops and the total number of printed character grows proportionally to n^2 .

Space Complexity: All have $O(1)$ auxiliary space because they use only a few loop variables and do not allocate extra memory based on n .

MODULE 2

Time and Space complexity of Searching and Sorting Algorithms.

Best Case	Average Case	Worst Case	Space Complexity	Brief Explanation
$O(1)$	$O(n)$	$O(n)$	$O(1)$	checks elements one by one. Best case if the element is first worst if it is last or absent.
$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$ $O(\log n)$	Repeatedly halves the search space. Works only on sorted arrays.
$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Repeatedly swaps adjacent elements. Largest element moves to the end after each pass.

MODULE 3

Best Case	Avg Case	Worst Case	Space Complexity	Space Complexity
$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	divides the array into halves sorts each half then merges them performance is always $O(n \log n)$

$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
---------------	---------------	----------	-------------

chooses a pivot and partitions the array around it. Very fast in practice, but worst when the pivot is always the smallest or largest element.

MODULE 4

A1) Time Complexity $O(n)$

The for loop runs from 1 to N ,
printing one number in each iteration.
Since there are N iterations the
running time grows linearly with N .
Space complexity.

$O(1)$
The program uses only a few variables
(N and i) regardless of the input
size.

A2) Time Complexity $O(n)$

Since the while executes N times.
Space complexity.

$O(1)$
The program uses only a fixed number
of variables
(N , sum and i). No extra memory.

Ans 3)

Time Complexity

$O(B)$

The loop ~~sums~~ runs B times and in each iteration one multiplication is performed.

Space Complexity

$O(1)$

The program uses only a few variables.





main.c...

Output



```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int N;
6      cin >> N;
7
8      for (int i = 1; i <= N; i++) {
9          cout << i << " ";
10     }
11
12     return 0;
13 }
```

Run



main.c...

Output



```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int N;
6      cin >> N;
7
8      int sum = 0;
9
10     for (int i = 1; i <= N; i++) {
11         sum += i;
12     }
13
14     cout << sum << endl;
15
16     return 0;
17 }
```

Run



main.c...

Output



```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int A, B;
6      cin >> A >> B;
7
8      long long result = 1;
9
10     for (int i = 1; i <= B; i++) {
11         result = result * A;
12     }
13
14     cout << result << endl;
15
16     return 0;
17 }
```

Run